

Question 1:

Compare FIFO (First-In, First-Out) with other page replacement algorithms (like LRU, LFU, and Optimal). Under what situations would FIFO be more efficient than other algorithms?

Solution:

FIFO vs. Other Page Replacement Algorithms

The **FIFO (First-In, First-Out)** page replacement algorithm is the simplest: it replaces the page that has been in memory the longest.

Feature	FIFO (First-In, First-Out)	LRU (Least Recently Used)	LFU (Least Frequently Used)	Optimal (MIN)
Replacement Rule	The oldest page in memory.	The page not used for the longest time.	The page with the smallest usage count.	The page that won't be used for the longest time in the future.
Implementation	Easy (uses a queue).	Complex (requires tracking time/stamps).	Complex (requires usage count tracking).	Impossible to implement in reality (requires future knowledge).
Performance	Can be poor; susceptible to Belady's Anomaly .	Generally very good; approximation of Optimal.	Varies; sometimes better than LRU, sometimes worse.	Best possible, serves as a benchmark.

Situations Where FIFO Can Be More Efficient

FIFO is rarely more efficient than LRU or Optimal, but it performs well in specific scenarios:

1. **Uniform Reference Pattern:** If processes access pages in a strictly **sequential** or **cyclic** order (e.g., iterating through a large array or a simple loop that touches all pages equally), the oldest page is also likely to be the next one needed least. In this case, FIFO's performance can approach that of LRU.
2. **Initial Loading:** During the initial phase of a process when pages are loaded for the first time, FIFO performs just as well as any other algorithm, as all pages cause a page fault initially.



Question 2:

Given a scenario where a system is experiencing thrashing, explain how the operating system can detect and address this problem. Discuss the strategies for reducing or eliminating thrashing, such as page swapping, working set management, and memory allocation tuning.

Solution:

Detecting and Addressing Thrashing

Thrashing occurs when a process spends more time **paging** (swapping pages between memory and disk) than executing instructions. This leads to a severe drop in CPU utilization and overall system performance.

Detection

The OS can detect thrashing by monitoring two key metrics:

- **Low CPU Utilization:** The CPU is mostly idle because processes are waiting for the disk I/O (page swaps) to complete.
- **High Paging Rate:** A very large number of page faults and page swaps per second, indicating constant disk activity.

When **paging rate is high** and **CPU utilization is low**, the system is likely thrashing.

Strategies for Reducing/Eliminating Thrashing

1. **Reduce the Degree of Multiprogramming (DoM):** The most direct solution is to **suspend** or **swap out** one or more processes. This reduces the contention for memory, allowing the remaining processes to get the physical frames they need to run efficiently. This temporarily increases the available memory for the remaining active processes.
2. **Working Set Management:**
 - The **Working Set** model determines the minimum set of pages a process needs to avoid thrashing. The OS estimates the working set for each process (the set of pages used over the last Δ time units).
 - **Strategy:** The OS ensures that only processes whose **working set can be completely resident in memory** are allowed to run. If the total working sets of all processes exceed the available physical memory, the DoM is reduced.
3. **Memory Allocation Tuning (Local vs. Global Replacement):**
 - **Local Replacement:** A process's page replacement only considers its own allocated frames. This helps **isolate** the impact of a thrashing process, preventing it from stealing frames from well-behaved processes.
 - **Global Replacement:** A process can replace any page in memory, regardless of which process owns it. Thrashing in one process can cause a system-wide slow down. Tuning involves adjusting the balance to ensure each process has a sufficient **minimum number of frames**.
4. **Page Swapping (Controlled Swapping):** While thrashing involves excessive **page** swapping, **process** swapping can be used to alleviate it.
 - If a process is continuously causing page faults (thrashing), the entire process can be **swapped out** to the disk to free up its allocated frames for other, more productive processes. It is only swapped back in when memory load is lower.



Question 3:

Explain the concept of non-contiguous memory allocation and how it overcomes the limitations of contiguous allocation. Discuss the techniques involved in non-contiguous memory allocation, focusing on Paging and Segmentation. How do these techniques help in minimizing fragmentation?

Solution:

Non-Contiguous Memory Allocation

Non-contiguous memory allocation is a memory management technique where a single process's **logical address space** is loaded into **non-adjacent physical memory blocks** (frames or segments).

Overcoming Contiguous Allocation Limitations

Contiguous allocation requires a process to occupy a single, contiguous block of physical memory. Its major limitations are:

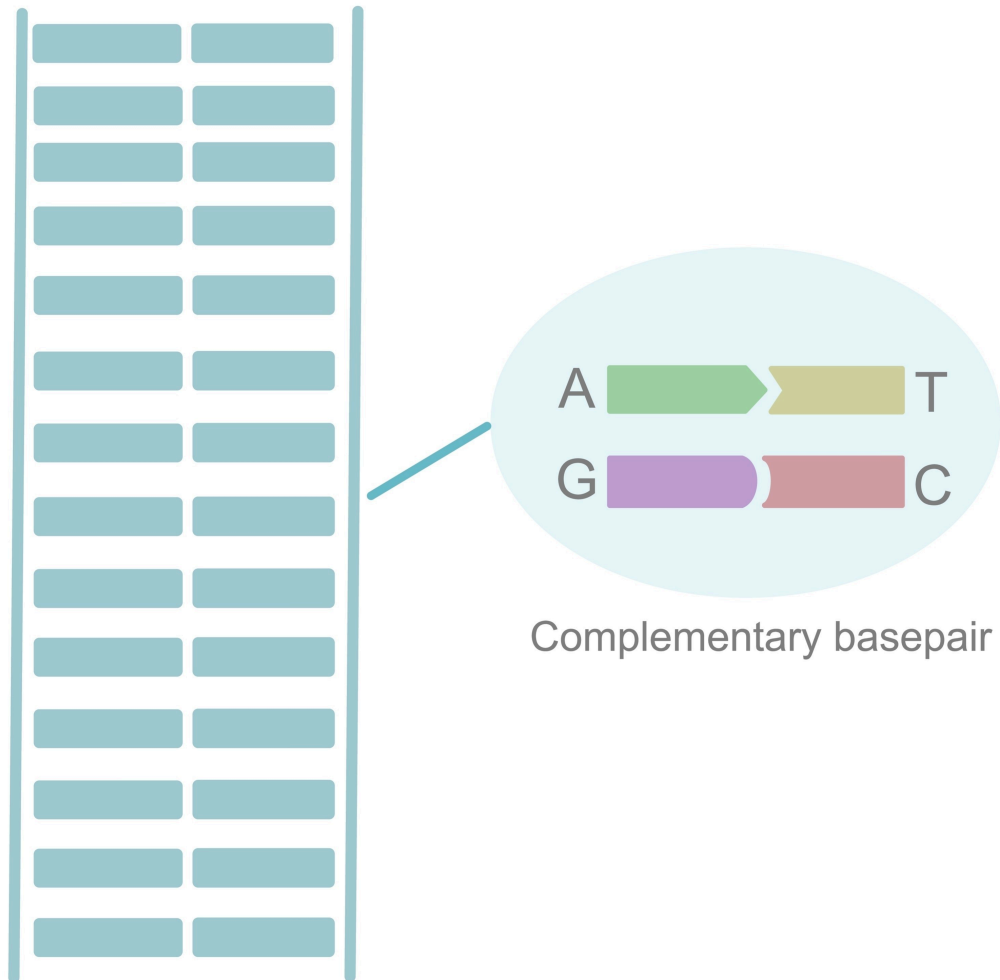
- **External Fragmentation:** Small, free memory holes are scattered throughout memory. The total free space might be large enough for a new process, but because it is not contiguous, the process cannot be loaded.
- **Rigidity:** Processes cannot easily grow or shrink, and memory partitioning must be determined in advance or dynamically with high overhead.

Non-contiguous allocation **solves external fragmentation** by allowing a process to use available free spaces wherever they are.

Techniques for Non-Contiguous Allocation

1. **Paging:**

- The logical address space of a process is divided into fixed-size blocks called **pages**.
- The physical memory is divided into fixed-size blocks of the same size called **frames**.
- Pages of a process can be loaded into any available frame in physical memory. A **Page Table** is used to map the process's pages to the physical frames.
- **Minimizing Fragmentation:** Paging eliminates external fragmentation entirely because any free frame can be used. It still suffers from **internal fragmentation**, as the last page of a process might not fill the entire frame.



Shutterstock

2. Segmentation:

- The logical address space is divided into a collection of variable-size blocks called **segments**. Segments correspond to logical units of the program (e.g., code, data, stack).
- Each segment is loaded into a single, contiguous block of physical memory, but the segments of a single process are **not contiguous** with each other. A **Segment Table** maps segment numbers to their base addresses and limits in physical memory.
- **Minimizing Fragmentation:** Segmentation eliminates the problem of forcing logical units to be equal-sized. However, since segments are of variable size, segmentation still

suffers from **external fragmentation**, although the holes are usually larger and fewer than in simple contiguous allocation.



Question 4:

Define segmented paging and explain how it combines the features of both segmentation and paging to manage memory. What are the benefits of segmented paging over pure segmentation and paging, and in what scenarios is segmented paging particularly useful?

Solution:

Segmented Paging

Segmented paging (or Paged Segmentation) is a hybrid memory management scheme that combines the logical view of **segmentation** with the physical memory management advantages of **paging**.

- **Combination:** A process's logical address space is first divided into **segments** (like in pure segmentation). However, each of these segments is then further divided into fixed-size **pages** (like in pure paging).
- **Address Translation:** A logical address is split into a **segment number** and a **page number and offset** within that segment.
 - The **Segment Table** entry points to a **Page Table** for that specific segment.
 - The segment's Page Table then maps the page number to the corresponding physical frame number, which is combined with the offset to form the final physical address.

Benefits

Over Pure Paging	Over Pure Segmentation
Logical View: Provides the user/programmer with a meaningful, logical view of memory (code, data, stack segments) for protection and sharing.	Eliminates External Fragmentation: By paging the segments, only fixed-size pages are allocated, eliminating external fragmentation.
Efficient Sharing and Protection: Sharing and protection can be applied at the logical segment level, instead of page by page.	Smaller Page Tables: Since each segment has its own page table, tables are generally smaller and more manageable than a single, huge page table for the entire process.

Useful Scenarios

Segmented paging is particularly useful in systems:

- That require robust **protection and sharing** of different logical parts of a program (e.g., sharing a read-only library code segment between multiple processes).
- Where **large, sparse address spaces** are common, as it allows for a more efficient representation of the address space compared to a single, monolithic page table.
- That need to **dynamically grow** different logical structures (like the stack or heap) without affecting others.



Question 5:

Discuss the concept of virtual memory and explain how it allows the system to run processes that may not fit entirely into physical memory. What role does demand paging play in virtual memory management? How does demand paging optimize memory usage, and what factors contribute to its performance?

Solution:

Virtual Memory Concept

Virtual memory is a memory management technique that separates the **user's logical memory** (virtual address space) from **physical memory** (RAM). This creates the illusion that a process has a large, contiguous memory space, potentially much larger than the physical memory available.

It achieves this by storing the entire logical address space of a process on secondary storage (**disk**) and only bringing the currently needed portions (pages) into physical memory. This allows:

- **Running Large Processes:** Processes can be larger than the physical memory size, as only a fraction of the process needs to be in RAM at any given time.
- **Increased Multiprogramming:** More processes can run concurrently because memory is used more efficiently; a process only occupies memory for the pages it is actively using.

Role of Demand Paging

Demand paging is the technique that makes virtual memory feasible and efficient.

- **Concept:** Pages are only loaded into physical memory **on demand**—that is, only when they are actually referenced by the CPU during execution. A process starts with **no pages** in memory.
- **Mechanism:** When the CPU tries to access a page that is not currently in memory, a **page fault** occurs. The OS catches the fault, locates the required page on the disk, reads it into a free physical frame, updates the page table, and then restarts the instruction that caused the fault.

Optimizing Memory Usage and Performance Factors

Demand paging optimizes memory usage by:

- **Low Memory Footprint:** Only the active, needed pages of a process reside in physical memory, leaving more RAM available for other processes.
- **Reduced I/O Time:** Less I/O is required at process startup, as the entire program doesn't need to be loaded upfront.

Factors contributing to its performance:

1. **Low Page Fault Rate:** The most critical factor. The lower the page fault rate, the less time is spent on slow disk I/O.

2. **Locality of Reference:** Demand paging relies heavily on the **principle of locality** (temporal and spatial). If a process accesses a small set of pages repeatedly for a period, the page fault rate remains low, leading to high performance.
3. **Frame Allocation:** Providing an adequate, but not excessive, number of physical frames to each process is essential. Too few frames can cause **thrashing**, while too many waste memory.
4. **Page Replacement Algorithm:** A good algorithm (like LRU or its approximations) minimizes the chance of replacing a page that will be needed soon, thus reducing future page faults.



Question 6:

Define thrashing in the context of virtual memory systems. What are the causes of thrashing, and how does it affect system performance? What strategies can be employed to reduce or prevent thrashing in an operating system?

Solution:

Thrashing Definition and Impact

Thrashing is a pathological condition in a virtual memory system where a process spends more time **paging** (swapping pages between main memory and secondary storage) than it does **executing** instructions. This results in the system doing almost no useful work.

Causes of Thrashing

1. **Insufficient Physical Memory (Low Frame Allocation):** The most common cause is that the sum of the **Working Sets** (the minimum set of pages required by active processes) of all running processes exceeds the total available physical memory.
2. **High Degree of Multiprogramming (DoM):** If too many processes are loaded into memory simultaneously, each receives too few frames. This low frame count means a process cannot hold all the pages it needs for its immediate execution locality, leading to frequent page faults.

3. **Poor Locality of Reference:** If a process frequently jumps randomly across its address space, accessing pages that are rarely or never adjacent, it quickly replaces useful pages, causing a continuously high page fault rate.

Effect on System Performance

Thrashing severely degrades system performance:

- **Low CPU Utilization:** The CPU is mostly idle because processes are constantly waiting for the disk I/O (page swaps) to complete.
- **High Paging/I/O Rate:** The paging disk is constantly busy, saturating the I/O channel.
- **System Collapse:** The OS, observing the low CPU utilization, mistakenly tries to increase DoM by adding even more processes, which only worsens the thrashing and can lead to a complete system collapse.

Strategies to Reduce/Prevent Thrashing

1. **Working Set Model/PFF (Page-Fault Frequency) Strategy:** Use a technique to determine the optimal number of frames needed by a process.
 - **Working Set:** Estimate the minimum frames needed and only allow processes whose working set can fit in memory to run.
 - **PFF:** Monitor the page fault rate. If the rate is too high (indicating thrashing), **increase** the number of frames allocated to the process. If the rate is too low, **decrease** the number of frames.
2. **Reduce the Degree of Multiprogramming (DoM):** When thrashing is detected, the OS should **suspend** or **swap out** one or more low-priority processes to free up frames for the remaining active processes.
3. **Local Page Replacement:** Use a local replacement algorithm, where a process can only replace a page within its own allocated frames. This prevents a thrashing process from stealing frames from well-behaved processes, isolating the problem.

Question 7:

Discuss address binding in the context of loading and linking, explaining the different types of address binding (compile-time, load-time, and execution-time). How do these bindings affect the execution of a process?

Solution:

Address Binding

Address binding is the process of mapping the **logical addresses** (or virtual addresses) used by a program in its source code to the corresponding **physical addresses** in main memory. This mapping is crucial for executing a process.

The binding can occur at three different stages:

1. **Compile-Time Binding:**

- **Mechanism:** If the process's physical memory location is known in advance (e.g., the OS will always load the program at address 0x0000), the compiler generates **absolute code** (physical addresses).
- **Effect on Execution:** Execution is very fast and efficient.
- **Limitation:** If the starting address changes, the code must be recompiled. It is rarely used in modern multiprogramming systems.

2. **Load-Time Binding:**

- **Mechanism:** If the starting address is not known at compile time, the compiler generates **relocatable code**. The **loader** then performs the binding by assigning final physical addresses as the program is loaded into memory.
- **Effect on Execution:** The process can be loaded anywhere in physical memory. Once loaded, it cannot be easily moved to a different location during execution.

3. **Execution-Time Binding (Run-Time Binding):**

- **Mechanism:** Binding is delayed until the process is running. The program generates logical addresses, and a dedicated hardware unit, the **Memory Management Unit (MMU)**, translates the logical addresses to physical addresses on the fly, for **every memory access**. A **base register** (or a complex mechanism like paging/segmentation) is used for the translation.
- **Effect on Execution:** This is the most flexible method. The process can be **swapped out and moved** to a different location in physical memory during its execution (essential for virtual memory). This flexibility comes with a slight performance overhead for the hardware translation.

Question 8:

In memory management, allocation strategies play a crucial role in determining how memory is assigned to processes. Compare and contrast the First Fit, Best Fit, and Worst Fit strategies for memory allocation. How do these strategies impact the efficiency of memory usage and process execution?

Solution:

Memory Allocation Strategies (First Fit, Best Fit, Worst Fit)

These three strategies are used in contiguous memory allocation (or segmentation) to decide which free block (hole) to allocate to a waiting process.

Strategy	Description	Impact on Efficiency & Fragmentation
First Fit	Allocates the first free block found that is large enough to satisfy the request. It stops searching immediately.	Efficiency: Fast allocation time, as it requires minimal searching.
		Fragmentation: Tends to leave large holes at the end of memory, but can generate many small, unusable holes near the beginning, leading to external fragmentation .
Best Fit	Allocates the smallest free block that is <i>just</i> big enough to satisfy the request. Requires searching the entire list of holes.	Efficiency: Slowest allocation time due to the exhaustive search.
		Fragmentation: Generates the smallest possible leftover hole (internal fragment) for the allocated space. However, it leads to a very large number of tiny, unusable external fragments .
Worst Fit	Allocates the largest free block available. Requires searching the entire list of holes.	Efficiency: Slow allocation time due to the exhaustive search.
		Fragmentation: Tries to create a large leftover hole to be more useful for future large requests.

Strategy	Description	Impact on Efficiency & Fragmentation
		In practice, it often leads to the same problem as Best Fit, leaving no genuinely large holes and creating moderate-sized, yet unusable, external fragments.

Overall Impact

- **Time Complexity:** First Fit is generally the fastest allocation strategy. Best Fit and Worst Fit are slower because they require a complete search of the available memory blocks.
- **Fragmentation:** Simulations show that **First Fit** and **Best Fit** generally perform better than Worst Fit in terms of memory utilization, despite the issues with external fragmentation. Worst Fit is often the poorest performer in terms of utilizing available space efficiently over time.



Question 9:

Explain the concept of virtual memory and how it allows processes to exceed physical memory limits. Discuss the concept of demand paging and its impact on system performance. How is page fault handling managed in demand paging, and what are the factors influencing its efficiency?

Solution:

Virtual Memory and Exceeding Limits

Virtual memory is an abstraction layer that separates a program's logical address space from the physical memory space. It uses the hard disk as a temporary extension of RAM (known as the swap space or backing store).

It allows processes to exceed physical memory limits because:

- The logical address space created by the compiler/linker can be much larger than RAM.

- The OS only loads the small portion of a process's code and data that is **currently active** into physical memory. The rest remains on the disk.

Demand Paging and System Performance

Demand Paging is the key implementation technique for virtual memory. It states that a page is only brought into physical memory **when an executing process requires it** (i.e., "on demand").

- **Impact on Performance:** Demand paging generally **improves** system performance and memory utilization by:
 - Reducing the I/O needed for loading a process.
 - Allowing a higher degree of multiprogramming.
 - Reducing the initial load time of a process.
 - However, performance depends heavily on keeping the **page fault rate** low. High page fault rates can lead to **thrashing** and a massive performance drop.

Page Fault Handling

A **page fault** occurs when the CPU attempts to access a page marked as "not present" (invalid) in the page table. The OS manages this in the following steps:

1. The CPU traps to the OS (context switch).
2. The OS determines the cause of the trap (page fault).
3. The OS determines the logical address that caused the fault and locates the required page on the backing store.
4. The OS finds a **free frame** in physical memory.
 - If no frame is free, a **page replacement algorithm** (e.g., LRU) is used to select a victim frame, which is then written out to the disk if it has been modified (dirty bit is set).
5. The required page is read from the disk into the free frame.
6. The **Page Table** is updated to reflect the new physical frame number, and the **valid/invalid bit** is set to "valid."
7. The instruction that caused the page fault is restarted.

Factors Influencing Efficiency:

- **Page Fault Service Time:** The time it takes to handle a page fault is dominated by the **disk I/O latency** (reading the page). Minimizing this time is crucial.

- **Page Replacement Algorithm:** A well-chosen algorithm minimizes the future fault rate.
- **Frame Allocation:** The number of frames allocated per process (too few causes thrashing, too many wastes memory).



Question 10:

Describe the function of memory management in an operating system. Discuss the role of loading and linking in the memory management process. Also, explain address binding and how it occurs at different stages (compile-time, load-time, and execution-time), with examples of how it impacts process execution.

Solution:

Function of Memory Management

Memory management is the OS function responsible for coordinating and optimizing the use of the computer's primary memory (RAM). Its main goals are:

1. **Allocation and Deallocation:** Assigning memory to processes when they request it and reclaiming it when they finish.
2. **Protection:** Ensuring that a process cannot access the memory space of another process or the OS.
3. **Relocation:** Allowing processes to be loaded into any available memory location and potentially moved during execution.
4. **Efficiency:** Maximizing CPU utilization and throughput by allowing a high degree of multiprogramming (e.g., via virtual memory).

Role of Loading and Linking

1. Linking:

- **Function:** Combines all code and data segments needed by a program, including library routines, into a single binary executable file.

- **Role in Memory Management:** It resolves external references between different parts of the code, assigning **relative or logical addresses** within the program's address space.

2. Loading:

- **Function:** Moves the final binary executable file from the disk into the main memory (RAM) so that the CPU can execute it.
- **Role in Memory Management:** It is the final stage where the program begins its life in physical memory. The loader is responsible for the actual **address binding** if it is to be done at load-time.

Address Binding and Impact (Recap)

Address binding is the process of mapping logical addresses (used in the program) to physical addresses (in RAM).

Binding Stage	When It Happens	Impact on Execution	Example/Scenario
Compile-Time	Before the program is loaded.	Fast but Inflexible: Program must be recompiled if its physical starting location changes.	Early simple OS with fixed memory partitions.
Load-Time	During the loading of the program into memory.	Flexible Location: Program can be loaded anywhere but cannot be moved once execution starts.	Simple single-user system where the starting address is determined dynamically by the loader.
Execution-Time	While the process is running (by the MMU).	Most Flexible: Allows the process to be swapped, moved, or use virtual memory. Slight Overhead for hardware translation.	Modern OS using paging or segmentation (necessary for virtual memory).



Question 11:

Explain the concept of Belady's Anomaly and discuss how it applies to page replacement algorithms. Which algorithms are susceptible to Belady's Anomaly?

Solution:

Belady's Anomaly

Belady's Anomaly (also known as the FIFO anomaly) is a counter-intuitive phenomenon observed in some page replacement algorithms. It states that, for a given page reference string, **increasing the number of available physical memory frames can sometimes lead to an increase in the number of page faults.**

This behavior contradicts the expected outcome, which is that more memory should result in better performance (fewer faults).

Application to Page Replacement Algorithms

The anomaly arises because the decision to replace a page based purely on its age in memory (FIFO) does not correlate with its future use. When the frame count increases, a previously replaced page might have stayed in memory longer, but that older page might still be needed in the expanded set, leading to a fault where one previously didn't occur.

Susceptible Algorithms

Belady's Anomaly only applies to page replacement algorithms that do **not** use the **stack property**.

The most prominent algorithm susceptible to Belady's Anomaly is:

- **FIFO (First-In, First-Out):** Because FIFO only considers the time of loading and not the frequency or recency of use, increasing the memory size can disrupt the queue order, potentially replacing a page that would have otherwise remained in a smaller memory configuration.

Algorithms NOT Susceptible

Algorithms that obey the stack property (i.e., the set of pages in memory with m frames is always a subset of the pages in memory with $m + 1$ frames) are **immune** to Belady's Anomaly. These include:

- **LRU (Least Recently Used)**
- **Optimal (MIN)**

»

Question 12:

Consider the following reference string: 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5, and a frame size of 3. Calculate the page fault rate using FIFO. Explain your solution step-by-step.

Solution:

Page Fault Calculation using FIFO

- **Reference String (R):** 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5
- **Frame Size (F):** 3
- **Total References:** 12

Step	Reference	Frame 1 (FIFO)	Frame 2	Frame 3	Page Fault (PF)
1	4	4			Y
2	3	4	3		Y
3	2	4	3	2	Y
4	1	1	3	2	Y (4 is oldest)
5	4	1	4	2	Y (3 is oldest)
6	3	1	4	3	Y (2 is oldest)
7	5	5	4	3	Y (1 is oldest)
8	4	5	4	3	N
9	3	5	4	3	N
10	2	5	2	3	Y (4 is oldest)

Step	Reference	Frame 1 (FIFO)	Frame 2	Frame 3	Page Fault (PF)
11	1	1	2	3	Y (5 is oldest)
12	5	1	2	5	Y (3 is oldest)

Total Page Faults: 10

Calculation of Page Fault Rate

$$\text{Page Fault Rate} = \frac{\text{Total Page Faults}}{\text{Total References}} \times 100\%$$

$$\text{Page Fault Rate} = \frac{10}{12} \times 100\%$$

$$\text{Page Fault Rate} \approx 0.8333 \times 100\% \approx \mathbf{83.33\%}$$



Question 13:

Given a reference string: 4, 7, 6, 1, 7, 6, 1, 2, 7, 2 and a frame size of 3, calculate the number of page faults using FIFO. Show your calculations and explain the results.

Solution:

Page Fault Calculation using FIFO

- Reference String (R): 4, 7, 6, 1, 7, 6, 1, 2, 7, 2
- Frame Size (F): 3
- Total References: 10

Step	Reference	Frame 1 (FIFO)	Frame 2	Frame 3	Page Fault (PF)
1	4	4			Y
2	7	4	7		Y
3	6	4	7	6	Y

Step	Reference	Frame 1 (FIFO)	Frame 2	Frame 3	Page Fault (PF)
4	1	1	7	6	Y (4 is oldest)
5	7	1	7	6	N
6	6	1	7	6	N
7	1	1	7	6	N
8	2	1	2	6	Y (7 is oldest)
9	7	7	2	6	Y (1 is oldest)
10	2	7	2	6	N

Total Page Faults: 6

Explanation of Results

The algorithm incurred **6 page faults** out of 10 memory references.

- The first **3 faults** (4, 7, 6) were necessary to fill the initially empty frames.
- The **4th fault** (1) replaced 4, which had been in memory the longest.
- The next **3 references** (7, 6, 1) found their pages in memory (hits).
- The **5th fault** (2) replaced 7, the oldest page at that time.
- The **6th fault** (7) replaced 1, the oldest page at that time, and brought 7 back into memory.
- The final reference (2) was a hit.



Question 14:

Given the following reference string: 3, 1, 2, 3, 4, 2, 3, 0, 3, 1, 3, and a frame size of 3, calculate the number of page faults using the LRU (Least Recently Used) algorithm.

Solution:

Page Fault Calculation using LRU

- **Reference String (R):** 3, 1, 2, 3, 4, 2, 3, 0, 3, 1, 3
- **Frame Size (F):** 3
- **Total References:** 11

Step	Reference	Frame 1	Frame 2	Frame 3	LRU Page to Replace	Page Fault (PF)
1	3	3			-	Y
2	1	3	1		-	Y
3	2	3	1	2	-	Y
4	3	3	1	2	-	N
5	4	4	1	2	1 (LRU is 1)	Y
6	2	4	1	2	-	N
7	3	4	3	2	4 (LRU is 4)	Y
8	0	4	3	0	2 (LRU is 2)	Y
9	3	4	3	0	-	N
10	1	1	3	0	4 (LRU is 4)	Y
11	3	1	3	0	-	N

Total Page Faults: 7



Question 15:

Consider the following reference string: 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5, and a frame size of 3. Calculate the page fault rate using LRU. Explain your solution step-by-step.

Solution:

Page Fault Calculation using LRU

- **Reference String (R):** 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5
- **Frame Size (F):** 3

- **Total References:** 12

Step	Reference	Frame 1	Frame 2	Frame 3	LRU Page to Replace	Page Fault (PF)
1	4	4			-	Y
2	3	4	3		-	Y
3	2	4	3	2	-	Y
4	1	1	3	2	4 (LRU is 4)	Y
5	4	1	4	2	3 (LRU is 3)	Y
6	3	1	4	3	2 (LRU is 2)	Y
7	5	5	4	3	1 (LRU is 1)	Y
8	4	5	4	3	-	N
9	3	5	4	3	-	N
10	2	2	4	3	5 (LRU is 5)	Y
11	1	2	4	1	3 (LRU is 3)	Y
12	5	5	4	1	2 (LRU is 2)	Y

Total Page Faults: 10

Calculation of Page Fault Rate

$$\text{Page Fault Rate} = \frac{\text{Total Page Faults}}{\text{Total References}} \times 100\%$$

$$\text{Page Fault Rate} = \frac{10}{12} \times 100\%$$

$$\text{Page Fault Rate} \approx 0.8333 \times 100\% \approx \mathbf{83.33\%}$$

(Note: For this specific reference string and frame size, LRU and FIFO yielded the same high number of page faults, indicating a poor locality of reference for this sequence.)



Question 16:

Given the following reference string: 1, 2, 3, 4, 2, 5, 3, 4, 2, 6, 3, 8, 4, and a frame size of 3, calculate the number of page faults using the Optimal page replacement algorithm.

Solution:

Page Fault Calculation using Optimal Algorithm

- **Reference String (R):** 1, 2, 3, 4, 2, 5, 3, 4, 2, 6, 3, 8, 4
- **Frame Size (F):** 3
- **Total References:** 13

The **Optimal** (or MIN) algorithm replaces the page in memory that will **not be used for the longest period of time** in the future.

Step	Reference	Frame 1	Frame 2	Frame 3	Optimal Page to Replace	Page Fault (PF)
1	1	1			-	Y
2	2	1	2		-	Y
3	3	1	2	3	-	Y
4	4	4	2	3	1 (Used furthest at ref 11)	Y
5	2	4	2	3	-	N
6	5	4	2	5	3 (Used furthest at ref 11)	Y
7	3	4	2	3	5 (Used furthest at ref 13)	Y
8	4	4	2	3	-	N
9	2	4	2	3	-	N
10	6	6	2	3	4 (Used furthest at ref 13)	Y
11	3	6	2	3	-	N
12	8	6	8	3	2 (Used furthest at ref 13)	Y
13	4	4	8	3	6 (Used furthest at ref 13)	Y

Total Page Faults: 9

Question 17:

Given the following reference string: 5, 3, 1, 5, 4, 1, 6, 3, 4, 5, 1, and a frame size of 3, calculate the number of page faults using the Optimal page replacement algorithm.

Solution:

Page Fault Calculation using Optimal Algorithm

- **Reference String (R):** 5, 3, 1, 5, 4, 1, 6, 3, 4, 5, 1
- **Frame Size (F):** 3
- **Total References:** 11

Step	Reference	Frame 1	Frame 2	Frame 3	Optimal Page to Replace	Page Fault (PF)
1	5	5			-	Y
2	3	5	3		-	Y
3	1	5	3	1	-	Y
4	5	5	3	1	-	N
5	4	5	3	4	1 (Used furthest at ref 11)	Y
6	1	1	3	4	5 (Used furthest at ref 10)	Y
7	6	1	6	4	3 (Used furthest at ref 8)	Y
8	3	1	6	3	4 (Used furthest at ref 9)	Y
9	4	1	6	4	6 (Used furthest at ref 10)	Y
10	5	5	6	4	1 (Used furthest at ref 11)	Y
11	1	1	6	4	5 (Used furthest at ref 10)	Y



Question 18:

Consider the following reference string: 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4, and a frame size of 4. Calculate the page fault rate using LRU. Explain your solution step-by-step.

Solution:

Page Fault Calculation using LRU

- **Reference String (R):** 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4
- **Frame Size (F):** 4
- **Total References:** 12

Step	Ref	F1	F2	F3	F4	LRU Page to Replace	Page Fault (PF)
1	0	0				-	Y
2	1	0	1			-	Y
3	2	0	1	2		-	Y
4	3	0	1	2	3	-	Y
5	0	0	1	2	3	-	N
6	1	0	1	2	3	-	N
7	4	0	1	2	4	3 (LRU is 3)	Y
8	0	0	1	2	4	-	N
9	1	0	1	2	4	-	N
10	2	0	1	2	4	-	N
11	3	3	1	2	4	0 (LRU is 0)	Y
12	4	3	1	2	4	-	N

Calculation of Page Fault Rate

$$\text{Page Fault Rate} = \frac{\text{Total Page Faults}}{\text{Total References}} \times 100\%$$

$$\text{Page Fault Rate} = \frac{6}{12} \times 100\%$$

$$\text{Page Fault Rate} = 0.5 \times 100\% = \mathbf{50\%}$$

»

Question 19:

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order. Perform the allocation of processes using:

- a) First Fit Algorithm
- b) Best Fit Algorithm
- c) Worst Fit Algorithm

Solution:

Memory Allocation Strategies

Available Partitions (Holes) (in KB): 200, 400, 600, 500, 300, 250

Processes (in KB) (Order): $P_1(357)$, $P_2(210)$, $P_3(468)$, $P_4(491)$

a) First Fit Algorithm

- **Rule:** Allocate the **first partition** that is large enough.

Process	Size (KB)	Partitions (Initial)	Partition Allocated (First Fit)	Remaining Partitions (KB)
P_1	357	200, 400, 600, 500, 300, 250	400	200, (400 - 357)=43, 600, 500, 300, 250
P_2	210	200, 43, 600, 500, 300, 250	600	200, 43, (600 - 210)=390, 500, 300, 250
P_3	468	200, 43, 390, 500, 300, 250	500	200, 43, 390, (500 - 468)=32, 300, 250
P_4	491	200, 43, 390, 32, 300, 250	None	P_4 cannot be allocated (largest available is 390).

Result (First Fit): P_1, P_2, P_3 are allocated. P_4 is waiting.



b) Best Fit Algorithm

- Rule:** Allocate the **smallest partition** that is large enough.

Process	Size (KB)	Partitions (Initial)	Partition Allocated (Best Fit)	Remaining Partitions (KB)
P_1	357	200, 400, 600, 500, 300, 250	400	200, (400 - 357)=43, 600, 500, 300, 250
P_2	210	200, 43, 600, 500, 300, 250	250	(200), 43, 600, 500, 300, (250 - 210)=40
P_3	468	200, 43, 600, 500, 300, 40	500	200, 43, 600, (500 - 468)=32, 300, 40
P_4	491	200, 43, 600, 32, 300, 40	600	200, 43, (600 - 491)=109, 32, 300, 40

Result (Best Fit): All processes are allocated.



c) Worst Fit Algorithm

- **Rule:** Allocate the **largest partition** available.

Process	Size (KB)	Partitions (Initial)	Partition Allocated (Worst Fit)	Remaining Partitions (KB)
P_1	357	200, 400, 600 , 500, 300, 250	600	200, 400, (600 - 357)= 243 , 500, 300, 250
P_2	210	200, 400, 243, 500 , 300, 250	500	200, 400, 243, (500 - 210)= 290 , 300, 250
P_3	468	200, 400 , 243, 290, 300, 250	None	P_3 cannot be allocated (largest available is 400).

Result (Worst Fit): P_1 and P_2 are allocated. P_3 and P_4 are waiting.

»

Question 20:

Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find the number of page faults using the Optimal Page Replacement Algorithm.

Solution:

Page Fault Calculation using Optimal Algorithm

- **Reference String (R):** 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3
- **Frame Size (F):** 4
- **Total References:** 14

Step	Reference	F1	F2	F3	F4	Optimal Page to Replace	Page Fault (PF)
1	7	7				-	Y

Step	Reference	F1	F2	F3	F4	Optimal Page to Replace	Page Fault (PF)
2	0	7	0			-	Y
3	1	7	0	1		-	Y
4	2	7	0	1	2	-	Y
5	0	7	0	1	2	-	N
6	3	3	0	1	2	7 (Used furthest at ref 14)	Y
7	0	3	0	1	2	-	N
8	4	3	0	4	2	1 (Used furthest at ref 14)	Y
9	2	3	0	4	2	-	N
10	3	3	0	4	2	-	N
11	0	3	0	4	2	-	N
12	3	3	0	4	2	-	N
13	2	3	0	4	2	-	N
14	3	3	0	4	2	-	N

Total Page Faults: 6



Question 21:

Consider the following reference string: 1, 2, 3, 4, 5, 1, 2, 3, and a frame size of 3. Calculate the page fault rate using FIFO, LRU, Optimal, and LFU. Compare the results and explain the differences.

Solution:

1. Calculation of Page Faults

- **Reference String (R):** 1, 2, 3, 4, 5, 1, 2, 3

- **Frame Size (F):** 3
- **Total References:** 8

a) FIFO (First-In, First-Out)

Ref	F1 (Oldest)	F2	F3	PF
1	1			Y
2	1	2		Y
3	1	2	3	Y
4	4	2	3	Y (Replaced 1)
5	4	5	3	Y (Replaced 2)
1	4	5	1	Y (Replaced 3)
2	2	5	1	Y (Replaced 4)
3	2	3	1	Y (Replaced 5)

Total FIFO Page Faults: 8

↻

b) LRU (Least Recently Used)

Ref	F1	F2	F3	LRU Page	PF
1	1			-	Y
2	1	2		-	Y
3	1	2	3	-	Y
4	4	2	3	1	Y (Replaced 1)
5	4	5	3	2	Y (Replaced 2)
1	4	5	1	3	Y (Replaced 3)
2	2	5	1	4	Y (Replaced 4)
3	2	5	3	5	Y (Replaced 5)

Total LRU Page Faults: 8

c) Optimal (MIN)

Look Ahead:

Ref	F1	F2	F3	Optimal Page	PF
1	1			-	Y
2	1	2		-	Y
3	1	2	3	-	Y
4	4	2	3	1 (Next use at ref 6)	Y
5	4	5	3	2 (Next use at ref 7)	Y
1	1	5	3	4 (Never used again)	Y
2	1	2	3	5 (Never used again)	Y
3	1	2	3	-	N

Total Optimal Page Faults: 7

d) LFU (Least Frequently Used)

Pages are replaced based on the lowest current usage count. Ties are broken by the oldest page (FIFO).

Ref	F1 (Count)	F2 (Count)	F3 (Count)	LFU Page	PF
1	1 (1)			-	Y
2	1 (1)	2 (1)		-	Y
3	1 (1)	2 (1)	3 (1)	-	Y

Ref	F1 (Count)	F2 (Count)	F3 (Count)	LFU Page	PF
4	4 (1)	2 (1)	3 (1)	1 (Count 1, Oldest)	Y
5	4 (1)	5 (1)	3 (1)	2 (Count 1, Oldest)	Y
1	4 (1)	5 (1)	1 (2)	3 (Count 1, Oldest)	Y
2	4 (1)	2 (2)	1 (2)	5 (Count 1, Oldest)	Y
3	3 (2)	2 (2)	1 (2)	4 (Count 1, Oldest)	Y

Total LFU Page Faults: 8

∞

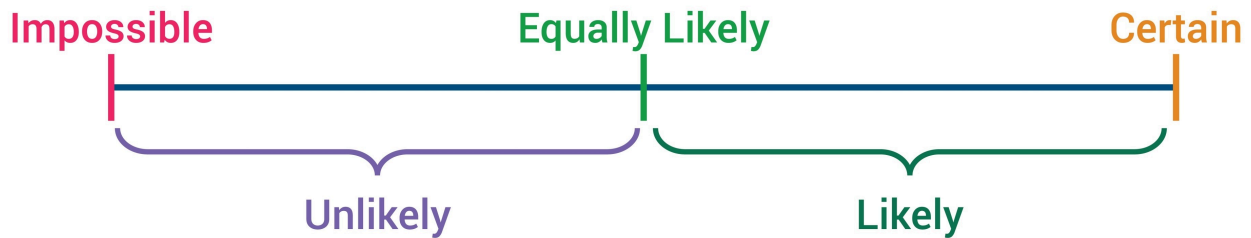
2. Comparison and Explanation

Algorithm	Total Page Faults	Page Fault Rate
FIFO	8	$8/8 = 100\%$
LRU	8	$8/8 = 100\%$
Optimal	7	$7/8 = 87.5\%$
LFU	8	$8/8 = 100\%$

Explanation of Differences

1. **High Fault Rate (8/8):** The reference string 1, 2, 3, 4, 5, 1, 2, 3 exhibits **poor locality of reference**. The sequence repeatedly accesses pages outside the current frame set, meaning every reference after the initial three results in a fault for FIFO, LRU, and LFU. The page needed next is always the one just replaced.

The Probability Scale



2. Optimal Superiority (7 Faults):

- The **Optimal** algorithm, by looking into the future, is able to identify page 3 (at reference 8) as a **Hit**.
- At reference 7 (page 2), the Optimal algorithm correctly determines that **page 5** will not be used again, replacing it to make room for page 2. This foresight allows page 3 to remain in memory, resulting in the only **Hit** during the replacement phase. Optimal always yields the minimum number of faults.

3. FIFO, LRU, LFU Performance:

- In this specific scenario, **FIFO, LRU, and LFU all perform poorly** (8 faults).
- The sequential, non-repeating pattern in the middle (4, 5) defeats LRU, as the least recently used page is always a necessary page.
- LFU also fails because all pages in the frame have the same low count (1) when a replacement is needed, causing it to effectively fall back to a near-FIFO replacement when ties are broken by age.